Introduction to Microcontrollers

Facilitator Guide

Prepared by BLIS GLOBAL Robotics Department

Copyright @2024

Week 6: Actuators and Output Devices

Introduction

1. Introduction to Actuators

Actuators are components that convert electrical energy into mechanical motion. They play a crucial role in robotics and automation systems by enabling robots to move, perform tasks, or interact with their environment. Actuators can be classified based on the type of motion they produce or the type of energy they use.

Types of Actuators:

1. **Motors:**
    - **DC Motors:** Direct Current (DC) motors provide continuous rotational motion and are widely used in robotics to drive wheels or rotating parts.
    - **Stepper Motors:** These provide precise control of angular position by rotating in discrete steps. They are commonly used in CNC machines and 3D printers.
2. **Servo Motors:**
    - A servo motor is a type of motor that allows precise control of angular position. It is commonly used in applications requiring exact movements, such as robotic arms, camera gimbals, and model airplanes.
3. **Solenoids:**
    - Solenoids create linear motion and are often used in locking mechanisms, switches, and valves.
4. **Linear Actuators:**
    - Linear actuators move in a straight line, making them ideal for pushing or pulling applications like automatic doors, robotic arms, or elevators.

---

2. Controlling Actuators with Arduino

The Arduino can control different types of actuators by sending electrical signals. Actuators often require either digital or Pulse Width Modulation (PWM) signals to control their motion.

DC Motors:

- A DC motor can be controlled using the Arduino by providing it with a voltage signal. The direction and speed of the motor can be managed using H-bridge motor drivers (like the L298N).

- To control speed, Arduino can use **PWM** (Pulse Width Modulation) signals. The analogWrite() function allows varying the voltage sent to the motor, controlling its speed.

Servo Motors:

- Servo motors are controlled by sending specific **PWM signals** that define the angular position of the motor's shaft.
- Servo motors typically have a range of 0 to 180 degrees and are controlled by sending a PWM signal corresponding to a specific angle.
- The **Servo library** in Arduino simplifies this control by providing functions to easily set the position of the servo.

Stepper Motors:

- Stepper motors require a sequence of electrical signals to rotate. Arduino can control these motors using libraries like **AccelStepper** to manage the sequence and speed.

---

3. Using a Servo Motor

A **servo motor** allows you to control its angle of rotation, which is useful for applications that require precise movement, such as steering mechanisms or robotic arms.

How Servo Motors Work:

- The servo motor has a control line, usually connected to an Arduino PWM pin.
- The Arduino sends a series of pulses, and the width of the pulses determines the position of the servo motor.
- The **Servo library** provides an easy way to send these pulses, making it simple to control the motor's position.

Servo Control with Arduino:

To control a servo motor with an Arduino, you connect the motor's signal pin to one of the Arduino's PWM pins, provide power, and ground the motor. The Arduino code will then set the position of the servo motor using the Servo library.

Arduino Code to Control a Servo Motor:

cpp Copy code

```cpp
#include <Servo.h>  // Include the Servo library

Servo myServo;      // Create a servo object
int pos = 0;        // Variable to store the servo position

void setup() {
  myServo.attach(9);  // Attach the servo to pin 9 on the Arduino
}

void loop() {
  // Move the servo from 0 to 180 degrees
  for (pos = 0; pos <= 180; pos += 1) {
    myServo.write(pos);  // Set the servo position
    delay(15);           // Wait for the servo to reach the position
  }

  // Move the servo back from 180 to 0 degrees
  for (pos = 180; pos >= 0; pos -= 1) {
    myServo.write(pos);  // Set the servo position
    delay(15);           // Wait for the servo to reach the position
  }
}
```

Explanation:

- **The servo motor moves from 0 degrees to 180 degrees and back to 0 degrees in small steps.**
- **The myServo.attach(9) command tells the Arduino to control the servo on pin 9.**
- **The myServo.write(pos) command sets the position of the servo motor, where pos is the angle in degrees.**

---

4. Hands-On Activity: Interface and Control a Servo Motor

In this hands-on activity, you will interface a servo motor with Arduino and control its position.

Materials Needed:

- **Arduino board (e.g., Uno)**
- **Servo motor**
- **Jumper wires**
- **Breadboard**
- **USB cable to connect Arduino to your computer**

Connecting the Servo Motor:

1. **Servo Signal Pin: Connect to pin 9 on the Arduino.**
2. **VCC Pin: Connect to the 5V pin on the Arduino.**
3. **GND Pin: Connect to the GND pin on the Arduino.**

---

5. Project: Automated Fan Control Using a Servo Motor

This project demonstrates the use of both sensors and actuators. You will create a system that uses a temperature sensor to read the temperature and control a fan (represented by a servo motor) based on the temperature readings.

Concept:

- **When the temperature exceeds a specific threshold (e.g., 30°C), the fan turns on (servo motor moves to a certain angle).**
- **When the temperature falls below the threshold, the fan turns off (servo motor moves to 0 degrees).**

Materials:

- **Arduino board**
- **Servo motor**
- **Temperature sensor (e.g., LM35)**
- **Breadboard and jumper wires**

Arduino Code for Automated Fan Control:

Copy code

```
Servo fanServo;      // Create a servo object for the fan
int tempPin = A0;    // Temperature sensor connected to analog pin A0
int threshold = 30;  // Temperature threshold to turn on the fan

void setup() {
  fanServo.attach(9);  // Attach the servo to pin 9
  Serial.begin(9600);  // Start serial communication
}

void loop() {
  int tempValue = analogRead(tempPin);  // Read the temperature sensor value
  float temperature = (tempValue * 5.0 * 100.0) / 1024;  // Convert the sensor value to Ce

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");

  // If the temperature is above the threshold, turn on the fan (servo)
  if (temperature > threshold) {
    fanServo.write(90);  // Set the servo to 90 degrees to simulate the fan turning on
    Serial.println("Fan ON");
  } else {
    fanServo.write(0);  // Set the servo to 0 degrees to simulate the fan turning off
    Serial.println("Fan OFF");
  }
```

Explanation:

- The temperature is read from the sensor and compared to the threshold.
- If the temperature exceeds the threshold, the servo motor moves to 90 degrees, simulating the fan turning on.
- If the temperature is below the threshold, the servo motor moves back to 0 degrees, simulating the fan turning off.

Summary:

- Actuators convert electrical signals into mechanical movement. Common types include DC motors, stepper motors, and servo motors.
- Servo motors provide precise control of angular positions and are controlled using PWM signals.
- Controlling actuators with Arduino requires understanding of how to interface them and how to write code that sends the right signals.
- In the hands-on activity, you learned how to control a servo motor and how to integrate sensors and actuators into a project, like the automated fan control system.

By mastering actuators, you gain the ability to make your robotic projects interactive and responsive to their environment.